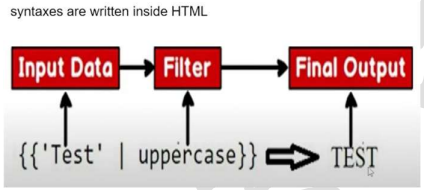
Pipes in Angular

What are Pipes?

Pipes formally known as filters, is a special operator in Angular template expressions that allows you to transform data declaratively in your template. Pipes let you declare a transformation function once and then use that transformation across multiple templates. Angular pipes use the vertical bar character (|).

Note that Angular's pipe syntax deviates from standard JavaScript, which uses the vertical bar character for the bitwise OR operator. Angular template expressions do not support bitwise operators.

Pipes in Angular are a feature that allow you to transform data in your application before displaying it to the user. Pipes are used to format, filter, and sort data and they can be used with both template-driven and reactive forms, as well as with other Angular components and services.



Types of Pipe

Angular pipes are categorized in two primary ways:

1. by their source (built-in or custom)
   - **Built-in Pipes:** Angular provides a collection of pre-built pipes for common data transformations for example: currency, date, etc.
   - **Custom Pipes:** When built-in pipes are insufficient for specific application needs, developers can create custom pipes by implementing the PipeTransform interface and decorating the class with @Pipe().
2. by their change detection behavior (pure or impure)
   - **Pure Pipes:** These are the default for both built-in and custom pipes.
     - They are executed only when Angular detects a pure change in the input value or parameters. A pure change is a change to a primitive input value (String, Number, Boolean, Symbol) or a change in an object reference (Date, Array, Function, Object).
     - Pure pipes are performant because they do not run on every change detection cycle, making them suitable for simple data transformations.
   - **Impure Pipes:** These pipes are executed during every change detection cycle, regardless of whether the input value or parameters have changed.
     - To create an impure pipe, you set the pure property to false in the @Pipe() decorator.
     - They are necessary for handling mutable data, such as detecting changes within an object or an array (e.g., adding an element to an existing array without changing the reference).
     - They should be used with caution as they can impact performance if not used appropriately.

To use a pipe in a template include it in an interpolated expression. Check out this example:

Sample.ts

```
import { JsonPipe, LowerCasePipe, SlicePipe, TitleCasePipe,
UpperCasePipe } from '@angular/common';

import { Component } from '@angular/core';

@Component({
  selector: 'app-sample',
  imports: [LowerCasePipe, UpperCasePipe, TitleCasePipe, SlicePipe,
JsonPipe],
  templateUrl: './sample.html',
  styleUrl: './sample.css',
})
export class Sample {
  msg: string = 'Hello, World!';
  cities: any[] = [
    { id: 101, name: 'Hyderabad' },
    { id: 102, name: 'Delhi' },
    { id: 103, name: 'Mumbai' },
    { id: 104, name: 'Kolkata' },
    { id: 105, name: 'Chennai' }]
}
```

Sample.html

```
<h2>Pipes Examples</h2>

Original Message : {{ msg }}<br />
In Lowercase : {{ msg | lowercase }}<br />
In Uppercase : {{ msg | uppercase }}<br />
In Titlecase : {{ msg | titlecase }}<br />

Slice first two chars : {{ msg | slice: 2 }}<br />
Slice chars- first two and after 8 : {{ msg | slice: 2 : 8 }}<br />

Array of Cities : {{ cities | json }}
```

Declare a num variable with a number assigned to it:

```
num: number = 123456.789;
```

in html:

```
<h2>Decimal Pipes Examples</h2>
<h3>123.4567890</h3>
<h3>num = {{ num }}</h3>
{{ 123.456789 | number }}<br />
{{ 123.456789 | number: '2.3-5' }}<br />
{{ 123.456789 | number: '4.2-5' }}<br />
{{ num | number: '2.1-2' }}<br />
{{ num / 100 | percent }}
```

Decimal Pipes Examples

123.4567890

123.457
123.45679
0,123.45679
32,423.23
32,423%

Declare amount variable in .ts file and assign a value to it and then use following in html:

```
<h3>Currency Pipes</h3>
Amount :
{{ amount }} <br />
{{ amount | currency }} <br />
{{ amount | currency : "USD" }} <br />
{{ amount | currency : "INR" }} <br />
{{ amount | currency : "GBP" }} <br />
{{ amount | currency : "INR" : true : "2.3-5" }} <br />
```

Currency Pipes

Amount : 5546.56
$5,546.56
$5,546.56
₹5,546.56
£5,546.56
₹5,546.560

Declare xdate variable in .ts and assign with current date:

```
xdate: Date = new Date(); // current date-time
```

Now in HTML:

```
<h3>Date Pipes</h3>
Date:
{{ xdate }}<br />
{{ xdate | date }}<br />
{{ xdate | date : "shortDate" }}<br />
{{ xdate | date : "longDate" }}<br />
{{ xdate | date : "fullDate" }}<br />
{{ xdate | date : "dd-MM-yyyy" }}<br />
```

Date Pipes

Date: Mon Mar 25 2024 22:18:59 GMT+0530 (India Standard Time)
Mar 25, 2024
3/25/24
March 25, 2024
Monday, March 25, 2024
25-03-2024

Few extra things you can try with Angular Pipes:

Combining multiple pipes in the same expression: You can apply multiple transformations to a value by using multiple pipe operators. Angular runs the pipes from left to right. The following example demonstrates a combination of pipes to display a localized date in all uppercase:

```
<p>The event will occur on {{ xdate | date | uppercase }}.</p>
```

Passing parameters to pipes : Some pipes accept parameters to configure the transformation. To specify a parameter, append the pipe name with a colon (:) followed by the parameter value.

For example, the DatePipe is able to take parameters to format the date in a specific way.

```
<p>The event will occur at {{ xdate | date: 'hh:mm' }}.</p>
```

Some pipes may accept multiple parameters. You can specify additional parameter values separated by the colon character (:). For example, we can also pass a second optional parameter to control the timezone.

```
<p>The event will occur at {{ xdate | date: 'hh:mm' : 'UTC'
}}[UTC].</p>
```

There are even more built-in pipes that you can use in your applications. You can find the list in the Angular documentation https://angular.dev/guide/templates/pipes.

In the case that the built-in pipes don't cover your needs, you can also create a custom pipe. Check out the next lesson to find out more.

Creating custom pipes

You can define a custom pipe by implementing a TypeScript class with the @Pipe decorator. A pipe must have two things:

- A name, specified in the pipe decorator
- A method named transform that performs the value transformation.

The TypeScript class should additionally implement the PipeTransform interface to ensure that it satisfies the type signature for a pipe.

Here is an example of a custom pipe that transforms strings to kebab case:

Go to terminal and use following command to generate a pipe:

>ng g p MyPipes\KebabCase

Write transforming code in kebab-case-pipe.ts as below:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'kebabCase',
})
export class KebabCasePipe implements PipeTransform {
  transform(value: string): string {
    return value.toLowerCase().replace(/ /g, '-');
  }
}
```

Declare a variable in component where you want to use a pipe on text:

```
sampleText: string = 'My Angular Application';
```

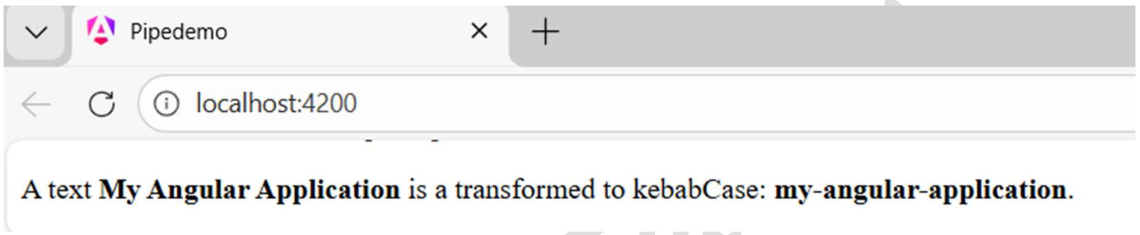
And now use pipe in any component as:

```
<p> A text <b>{{ sampleText }} </b>is a transformed to kebabCase:
<b>{{ sampleText | kebabCase }}</b>.</p>
```

Use respective import to destination:

```
import { KebabCasePipe } from "../../MyPipes/kebab-case-pipe";
```

Output:



Another example of custom pipes

In a HR / Payroll dashboard, salaries come from backend as plain numbers. But on UI, HR wants it in Indian currency format with label like: "₹ 4.5 LPA", "₹ 12.5 LPA". Make it reusable across multiple components.

750000 ➔ ₹ 7.5 LPA
1200000 ➔ ₹ 12.5 LPA
45000 ➔ ₹ 4.5 LPA

Built-in currency pipe cannot convert to LPA format, so we will create a custom pipe.

Go to terminal

>ng generate pipe shared/salaryLpa

In salary-lpa-pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'salaryLpa',
})
export class SalaryLpaPipe implements PipeTransform {
```

```
  transform(salary: number): string {
    if (!salary || salary <= 0) {
      return 'N/A';
    }

    const lpa = salary / 100000;
    return `₹ ${lpa.toFixed(1)} LPA`;
  }
}
```

To use let's create emplist component:

>ng g c components\emplist

In .ts file

```
import { Component } from '@angular/core';
import { SalaryLpaPipe } from '../../shared/salary-lpa-pipe';

@Component({
  selector: 'app-emplist',
  imports: [SalaryLpaPipe],
  templateUrl: './emplist.html',
  styleUrl: './emplist.css',
})
export class Emplist {
  employees: any[] = [
    { id: 101, name: 'Alice', salary: 750000 },
    { id: 102, name: 'Bob', salary: 1250000 },
    { id: 103, name: 'Charlie', salary: 450000 },
    { id: 104, name: 'David', salary: 550000 },
    { id: 105, name: 'Eve', salary: 950000 },
  ];
}
```

In HTML:

```
<table>
  <tr>
    <th>Name</th>
    <th>Salary</th>
  </tr>
  @for (emp of employees; track $index) {
```

```
    <tr>
      <td>{{ emp.name }}</td>
      <td>{{ emp.salary | salaryLpa }}</td>
    </tr>
  }
</table>
```

Output after using salaryLpa pipe:

Pipes Demo

Name	Salary
Alice	₹ 7.5 LPA
Bob	₹ 12.5 LPA
Charlie	₹ 4.5 LPA
David	₹ 5.5 LPA
Eve	₹ 9.5 LPA

DO IT YOURSELF

In an E-commerce / Order Management system, order status comes as raw strings and need to be displayed as:

- PENDING
- SHIPPED
- DELIVERED
- CANCELLED
- UNKNOWN

Order ID	Status
1	Pending
2	Shipped
3	Delivered
4	Cancelled
5	Unknown

Create a custom pipe for the same.

End of Chapter